

Tentamen Databases 26 maart 2013 (12:00-14:45)

Grading: starts at 1; question 1(a) gives 1.5 points, 3(a) and 3(b) give 1 point; the rest 0.5 points. Please answer all questions either in English (preferred) or in Dutch.

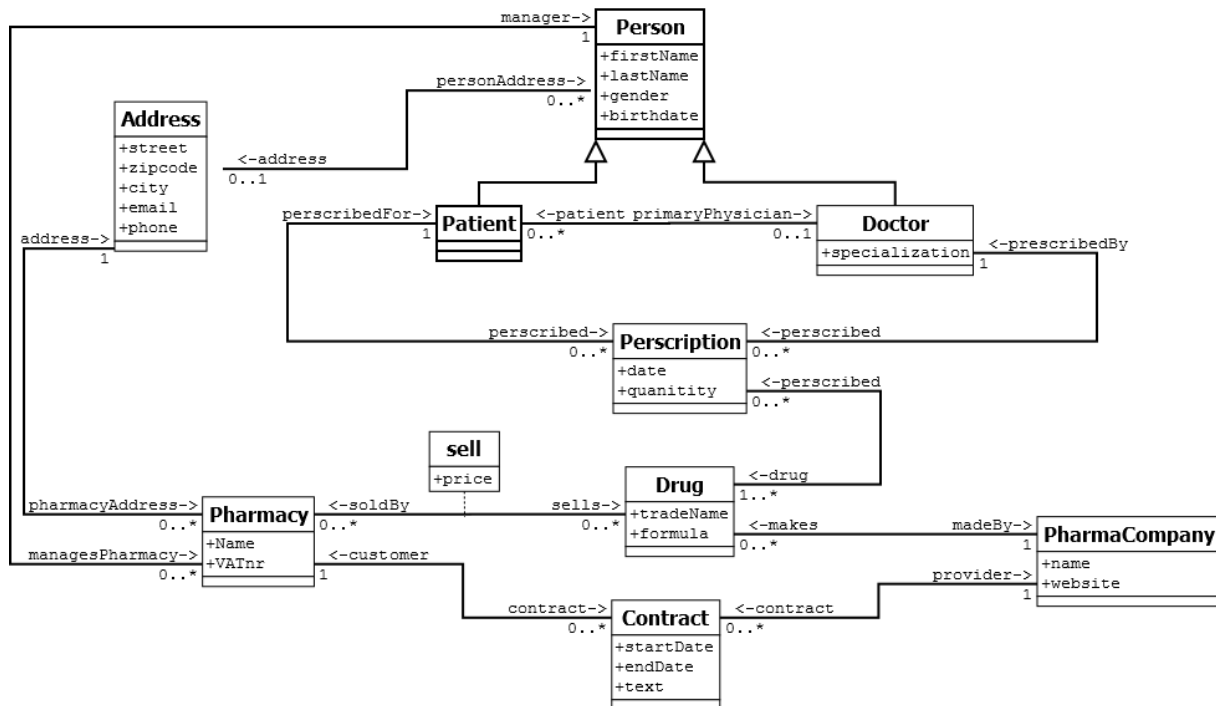
If you did the partial exam this year, then we will use your score there, if it is larger than the total you obtain on questions (1) and (2) in this exam (i.e. you might skip questions 1+2 now if your partial exam score is good).

1 Conceptual Design

Consider the following case: *We are modeling a chain of pharmacies. Patients have a first name, last name, gender and birthdate. We typically know the address of a person. An address consists of a street, zipcode, and city, and we also keep an email address and phone number. For doctors we keep the same data as for patients but that additionally have a specialization. Each pharmaceutical company is identified by name and website URL. For each drug, the trade name and formula must be recorded. Each drug is sold by a given pharmaceutical company, and the trade name identifies a drug uniquely from among the products of that company. Each pharmacy has a name, VAT number, address, and a manager. Every patient has a primary physician (i.e. doctor). Each pharmacy sells several drugs and has a price for each. A drug could be sold at several pharmacies, and the price could vary from one pharmacy to another. A prescription made by a doctor for a patient. Doctors prescribe drugs for patients. A doctor could prescribe one or more drugs for several patients, and a patient could obtain prescriptions from several doctors. Each prescription has a date and a quantity associated with it. Pharmaceutical companies have long-term contracts with pharmacies. A pharmaceutical company can contract with several pharmacies, and a pharmacy can contract with several pharmaceutical companies. For each contract, you have to store a start date, an end date, and the text of the contract.*

Question 1(a): Provide a conceptual database model in the form of a UML Class Diagram. Explain the most important design choices and document relevant assumptions.

Answer:



Question 1(b): Give the associated relational schema, indicating relations $R(A_1, \dots)$ with their attributes A_i , primary keys underlined, and foreign key relationships ($\rightarrow$). Separately also comment on NULLable attributes (if any), and constraints that a relational database could check, including candidate keys.

Answer:

Address(id, street, zipcode, city, email, phone)

// nullable all except id

Person(id, firstName, lastName, gender, birthDate, address $\rightarrow$ Address, specialization

primaryPhysician $\rightarrow$ Person) // nullable: address, specialization, primaryPhysician

PharmaCompany(id, name, website) // candidate key: name

Drug(id, company $\rightarrow$ PharmaCompany, tradeName, formula) // cand. key: (company, tradeName)

MakesDrug(company $\rightarrow$ PharmaCompany, drug $\rightarrow$ Drug)

Prescription(id, patient $\rightarrow$ Person, doctor $\rightarrow$ Person, drug $\rightarrow$ Drug, date, quantity)

Pharmacy(id, name, manager $\rightarrow$ Person, VATnr, address $\rightarrow$ Address)

SellsDrug(pharmacy $\rightarrow$ Pharmacy, drug $\rightarrow$ Drug, price)

Contract(id, pharmacy $\rightarrow$ Pharmacy, company $\rightarrow$ PharmaCompany, startDate, endDate, text)

We should not subtract points for missing constraints, as this was never emphasised during exercises or assignments. Nullables should be annotated though.

2 Normalization

Given $R(A, B, C, D, E, F)$, and FDs: $f = \{ AE \rightarrow DEF, C \rightarrow DE, F \rightarrow CD \}$

Show your intermediate steps in all the answers.

Question 2(a): is f canonical (a minimal basis)? If not, make it so.

Question 2(b): what are the minimal (candidate) keys?

Question 2(c): is R in BCNF already? If not, make it so, and tell if any FD's get lost.

Question 2(d): is R in 3NF already? If not, make it so.

Answer:

(a) The canonical FD algorithm would split $AE \rightarrow DEF$ into $AE \rightarrow D$, $AE \rightarrow E$ and $AE \rightarrow F$, of which $AE \rightarrow E$ is trivial and gets deleted. Also $C \rightarrow DE$ becomes $C \rightarrow D$ and $C \rightarrow E$; and $F \rightarrow CD$ becomes $F \rightarrow C$ and $F \rightarrow D$. Left-hand side elimination in the remaining FDs is only relevant for $AE \rightarrow C$ and $AE \rightarrow D$ but delivers nothing new. That is, E cannot be dropped because A^+ is only A , and A cannot be dropped because E^+ is just E . The final question is if any FD can be dropped. Now, $AE \rightarrow D$ can be dropped because $AE \rightarrow F \rightarrow D$. Also, $F \rightarrow D$ can be dropped, because $F \rightarrow C \rightarrow D$. So a (the) canonical set of FDs is $f = \{ AE \rightarrow F, C \rightarrow DE, F \rightarrow C \}$.

(b) Using the optimized version of the candidate key algorithm, we note that A and B are not on a right hand side in $f = \{ AE \rightarrow F, C \rightarrow DE, F \rightarrow C \}$, so AB must be part of any key. It is our first candidate, but it is not a key, as $AB^+ = AB$. Thus we extend it into ABC , ABD , ABE , and ABF respectively. Now $ABC^+ = ABCDEF$, $ABD^+ = ABD$, $ABE^+ = ABCDEF$ and $ABF^+ = ABCDEF$. So ABC , ABE and ABF are minimal keys. We can extend ABD into $ABCD$ (superset of ABC), $ABDE$ (superset of ABE) and $ABDF$ (superset of ABF). These three candidates are all keys, but they are a superset of a minimal key and hence not minimal.

So, R has three minimal keys: ABC , ABE and ABF .

(c) R is not in BCNF because none of the lhs is a key.

We first maximize the right hand sides of the canonical set of FDs $f = \{ AE \rightarrow F, C \rightarrow DE, F \rightarrow C \}$ into $f'' = \{ AE \rightarrow CDF, C \rightarrow DE, F \rightarrow CDE \}$.

Then, we examine these FDs one by one and split.

There are two possible outcomes depending on the split order:

Split on $AE \rightarrow CDF$ first: $R_1(\underline{A}, \underline{B}, \underline{E})$, $R_2(\underline{A}, \underline{F})$, $R_3(\underline{F}, \underline{C})$, $R_4(\underline{C}, \underline{D}, \underline{E})$.

Otherwise: $R_1(\underline{A}, \underline{B}, \underline{F})$, $R_2(\underline{F}, \underline{C})$, $R_3(\underline{C}, \underline{D}, \underline{E})$.

Both decompositions lose the FD: $AE \rightarrow F$, NF_3 will not have this.

(d) this relation is not in 3NF because none of the lhs of the FDs is a key; and, the presence of D in $C \rightarrow DE$ is a problem, since D is not part of any minimal key.

3NF decomposition follows from the canonical FD set:

$R_1(\underline{A}, \underline{E}, \underline{F})$, $R_2(\underline{C}, \underline{D}, \underline{E})$, $R_3(\underline{F}, \underline{C})$, $R_4(\underline{A}, \underline{B}, \underline{C})$

R_4 was added because the R_1 - R_3 did not contain any original key. One could alternatively have added $R_4(\underline{A}, \underline{B}, \underline{E})$ or $R_4(\underline{A}, \underline{B}, \underline{F})$.

3 SQL

Consider the following schema:

Suppliers(sid,sname,saddress)

Parts(pid,pname,color)

Catalog(sid→Suppliers,pid→Parts,cost)

The primary keys are underlined and → Rel indicates a foreign key relationship with the primary key of Rel.

Formulate the following queries in SQL. You will only obtain maximal points for your answers if your answers avoid use of GROUP BY in favor of existential quantification.

Question 3(a): Find the names of the cheapest suppliers of part 42 (i.e. pid=42).

Answer:

```
SELECT C.name
FROM Suppliers S JOIN Catalog C ON S.sid = C.sid
WHERE C.pid=42
WHERE NOT EXISTS
  (SELECT "blah"
   FROM Catalog C1
   WHERE C1.pid = 42 AND C1.price < C.price)
```

Question 3(b): Find (pid,sid) of those suppliers and products who do not offer that product at lowest price.

Answer:

```
SELECT C.sid, C.pid
FROM Catalog C
WHERE EXISTS
  (SELECT C1.sid
   FROM Catalog C1
   WHERE C1.pid = C.pid AND C1.cost < C.cost)
```

4 Concurrency

Question 4(a): Describe the notions of a serial, serializable and conflict equivalent and conflict serializable schedules.

Answer:

- **Serial schedule:** A schedule S is **serial** if, for every transaction T participating in the schedule, all the operations of T are executed consecutively in the schedule.
- **Serializable schedule:** A schedule S is **serializable** if it is equivalent to some serial schedule of the same n transactions.
- **Conflict equivalent:** Two schedules are said to be conflict equivalent if the order of any two conflicting operations is the same in both schedules.
- **Conflict serializable schedule:** schedule is conflict equivalent with a serial schedule.

Question 4(b): Consider the following concurrent schedule. Is this schedule conflict serializable? If it is, give an equivalent serial schedule.

T ₀	T ₁	T ₂
		read(Y)
		read(Z)
read (X)		
writeX)		
		write(Y)
		write(Z)
	read(Z)	
read(Y)		
write(Y)		
	read(Y)	
	write(Y)	
	read(X)	
	write(X)	

Answer: this execution is conflict serializable, because its corresponding dependency graph does not have any cycles. An equivalent schedule would be to serially execute the transactions in order T₂→T₀→T₁

5 Database APIs

Question 5(a): describe three features of JDBC that are often used to tune the performance of applications.

Answer:

connection pooling
prepared statements
stored procedures
bulk insert

Question 5(b): explain what the term “impedance mismatch” refers to and discuss LinQ in this respect.

Answer:

Impedence mismatch refers to the difference in data models between a database system and the application program accessing the database. For instance, in databases values maybe NULL, in programming languages not. Also, the data types used in database systems do not necessarily match fully on the data types supported by the programming language.

LinQ is an extension to C# and Visual Basic that makes it possible to query any kind of collection; be it in database systems, or Lists, Arrays, etc. Because LinQ is integrated right into these programming languages, there is no impedance mismatch. LinQ offers

static type analysis and checking; something otherwise impossible for database access code.

6 Data Warehousing and OLAP

Question 6(a): Using the SQL 1999 GROUP BY CUBE extension, and assuming productName has 10 different values and shopCity has 5 different values (and all combinations occur); how many tuples will be returned by the query:

```
SELECT shopCity, productName, sum(price_paid)
FROM sales
GROUP BY CUBE(shopCity,ProductNames)
```

Answer:

50 (shopCity,ProductNames) combinations
5 (shopCity,NULL) combinations
10 (NULL,ProductNames) combinations
and (NULL,NULL)
for a total of 66 groups.

Question 6(b): Discuss some advantages and disadvantages of HBase (a key-value noSQL store).

Answer:

HBase offers unlimited scalability for inserting, reading and updating records on a cluster architecture. HBase, however does not offer a high-level query interface, just the Put()/Get() API. Complex queries e.g, with joins or aggregations are very hard to implement, therefore. HBase also does not offer full ACID properties (note: it actually offers a bit of ACID. Only on transactions inside the same RegionServer. And a transaction cannot be a combination of multiple Get() and Put() requests with client think time in between).